

8 de octubre de 2025



UNIVERSIDAD CATOLICA
DE TEMUCO

Laboratorio n°2: Introduccion a GitHUB y Latex

Nombre:

Franchesca Marcial

Profesor:

Guido Mellado

Ayudante:

Fernando Valdés

Índice

1. Explicaciones	3
1.1. herencia,clase abstracta,polimorfismo,interfaz.	3
2. Ejemplos en Python y diagramas.	3
2.1. Ejemplo de Herencia	3
2.2. Ejemplo de clase abstract	4
2.3. Ejemplo de Polimorfismo	4
2.4. Ejemplo de Interfaz	5
3. Method Resolution Order (MRO)	5
3.1. Ejemplo de MRO	5

1. Explicaciones

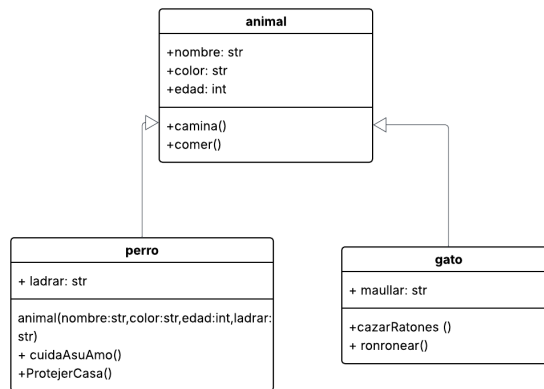
1.1. herencia, clase abstracta, polimorfismo, interfaz.

- herencia: La herencia es un tipo de relación entre tablas de entidad, donde existe la superclase y la clase hijo, donde la clase hijo 'hereda' todos los atributos y métodos de la superclase, pero aun as la clase hijo puede modificar y agregar nuevos atributos y métodos.
- clase abstracta: Se trata de una clase que no puede ser instanciada, eso quiere decir que no se pueden crear objetos directamente en ella, ya que más bien está diseñada para ser como clase plantilla para otras clases, como las @dataclass.
- polimorfismo: El polimorfismo es cuando tenemos una superclase y muchas clases hijos y todas las clases hijos heredan un método en común , por ejemplo método de hacer un ruido, en lugar de crear variables para cada tipo específico, usamos la clase base para referirnos a cualquier objeto que pertenezca a la jerarquía y el código solo llama a un solo método sin verificar el objeto que hace el ruido.
- interfaz: Es como un decorador, un contrato o un conjunto de reglas que una clase debe aceptar y cumplir, además, tampoco se pueden crear objetos directamente.

2. Ejemplos en Python y diagramas.

En esta sección se presentan ejemplos prácticos en lenguaje Python para ilustrar los conceptos de Herencia, Clases Abstractas, Polimorfismo e Interfaces. Cada ejemplo muestra cómo se aplican estos principios en la programación orientada a objetos (POO).

2.1. Ejemplo de Herencia



En este diagrama de ejemplo la clase perro y la clase gato heredan todos los atributos y metodos de su clasepadre llamada .animalz se puede ver que tambien cada clasehijo agrega su propio atributo y sus propios metodos.

2.2. Ejemplo de clase abstract

<<abstract>> animal
+nombre: str +color: str +edad: int
+animal(nombre:str,color:str,edad:int) +hacer_sonido(): <<abstract>> +moverse(): <<abstract>>

```
from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, nombre: str, color: str, edad: int):
        self.nombre = nombre
        self.color = color
        self.edad = edad

    @abstractmethod
    def hacer_sonido(self):
        pass

    @abstractmethod
    def moverse(self):
        pass
```

Este ejemplo ilustra cómo definir métodos abstractos que las subclases deberán implementar, garantizando que todas compartan una estructura común pero con comportamientos específicos.

2.3. Ejemplo de Polimorfismo

```
class Gato(Animal):
    def hablar(self):
        return "Miau!"

animales = [Perro("Firulais"), Gato("Michi"), Animal("Desconocido")]

for a in animales:
    print(f"{a.nombre}: {a.hacer_sonido()}")
```

El polimorfismo permite que diferentes objetos respondan de manera distinta al mismo método `hablar()`, sin necesidad de conocer su tipo específico.

2.4. Ejemplo de Interfaz

```
from abc import ABC, abstractmethod

class Volador(ABC):
    @abstractmethod
    def volar(self):
        pass

class Pajaro(Volador):
    def volar(self):
        return "El pájaro está volando"

class Avion(Volador):
    def volar(self):
        return "El avión despegó"

# Uso
objetos = [Pajaro(), Avion()]
for o in objetos:
    print(o.volar())
```

En este caso, la clase `Volador` actúa como una interfaz: define un conjunto de métodos que las clases hijas deben implementar.

3. Method Resolution Order (MRO)

El **MRO (Method Resolution Order)** define el orden en que Python busca los métodos y atributos en una jerarquía de herencia múltiple. Esto es importante cuando una clase hereda de varias clases al mismo tiempo.

3.1. Ejemplo de MRO

```
class A:
    def saludo(self):
        print("Hola desde A")

class B(A):
    def saludo(self):
        print("Hola desde B")

class C(A):
    def saludo(self):
        print("Hola desde C")

class D(B, C):
    pass

d = D()
d.saludo()
print(D.mro())
```

En este caso, Python imprimirá:

```
Hola desde B
[<class '__main__.D'>, <class '__main__.B'>,
 <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]
```

El resultado muestra que el orden de búsqueda de métodos sigue la secuencia: $D \rightarrow B \rightarrow C \rightarrow A \rightarrow \text{object}$. Esto significa que Python ejecuta el método `saludo()` de la clase B primero, respetando el MRO.